

CENG428 Neural Networks: Take-Home Practice Exam

Main framework: PyTorch and torchvision

Base dataset: MS COCO 2017

Main task: Detect or segment synthetic shapes added to natural images

1. Objective

In this practice exam, you will build a complete deep learning pipeline for detecting synthetic visual additions in natural images.

You will:

1. download and load the MS COCO 2017 dataset using `torchvision.datasets.CocoDetection` ,
2. add synthetic shapes to COCO images using your own code,
3. automatically generate labels for the added shapes,
4. train a CNN-based model to find the synthetic additions,
5. evaluate the model on a held-out test set,
6. compare your neural network against a simple baseline, and
7. report your results critically.

The main goal is not only to obtain high numerical performance, but also to demonstrate that you understand dataset construction, label generation, model training, evaluation, visualization, and experimental analysis.

2. Dataset Requirement

All students must use **MS COCO 2017** as the base natural-image dataset.

You must use the following torchvision dataset class:

```
torchvision.datasets.CocoDetection
```

COCO must be used only as a source of natural background images. The original COCO object labels are **not** the target labels for this assignment. Your target labels must come from the synthetic shapes that you add to the images.

You must use the following COCO files:

```
train2017/  
val2017/  
annotations/instances_train2017.json  
annotations/instances_val2017.json
```

Recommended directory structure:

```
data/  
  coco/  
    train2017/  
    val2017/  
    annotations/  
      instances_train2017.json  
      instances_val2017.json
```

Example initialization:

```
from torchvision.datasets import CocoDetection  
  
train_base = CocoDetection(  
    root="data/coco/train2017",  
    annFile="data/coco/annotations/instances_train2017.json"  
)  
  
val_base = CocoDetection(  
    root="data/coco/val2017",  
    annFile="data/coco/annotations/instances_val2017.json"  
)
```

You must then write your own PyTorch `Dataset` wrapper around `CocoDetection`. This wrapper should:

1. load a COCO image,
2. add one or more synthetic shapes,

3. generate the corresponding target label automatically,
 4. return the modified image and generated target.
-

3. Required Train / Validation / Test Split

To make results comparable across students, use the following fixed split protocol.

1. Use images from `train2017` for training.
2. Use images from `val2017` for validation and testing.
3. Sort COCO image IDs in increasing order.
4. Use the first **5,000** images from `train2017` as the required training subset.
5. Use the first **1,000** images from `val2017` as the validation set.
6. Use the next **1,000** images from `val2017` as the test set.

Students with stronger computing resources may use more than 5,000 training images, but they must still report results on the required fixed validation and test sets.

Validation and test synthetic shapes must be generated deterministically. Use the following global seed:

```
GLOBAL_SEED = 2025
```

Do **not** use Python's built-in `hash()` function for reproducible seeds, because its output may vary between sessions. Use a stable hash, for example:

```
import hashlib

def make_seed(split_name, image_id, global_seed=2025):
    key = f"{split_name}_{image_id}_{global_seed}".encode("utf-8")
    return int(hashlib.sha256(key).hexdigest()[:8], 16)
```

You may use random synthetic generation during training, but validation and test samples must be reproducible exactly.

4. Synthetic Shape Generation

For each selected image, generate a modified image by adding synthetic shapes. The synthetic shapes may include:

- circles,
- rectangles,
- triangles,
- ellipses,
- polygons,
- line segments,
- stars or simple icons.

Your generator must vary at least the following properties:

- shape type,
- location,
- size,
- color,
- opacity,
- rotation, where applicable,
- number of shapes per image.

Positive images should contain at least one synthetic shape. You must also include negative images with no target synthetic shape.

Recommended positive/negative ratio:

70% positive images
30% negative images

For positive images, use a random number of shapes, for example:

1 to 3 synthetic shapes per positive image

Your synthetic shapes should not make the task trivial. In particular, the problem should not be solvable by a simple color-thresholding rule. To make the task nontrivial, you must use at least **four** of the following difficulty mechanisms:

- random opacity,
- anti-aliased shape boundaries,
- random blur,
- additive noise,
- low-contrast colors,
- colors sampled from local or global image statistics,
- partial transparency,
- overlapping shapes,

- random resizing or cropping,
- hard negative images,
- distractor shapes that are not included in the target label.

You must visualize at least 12 generated examples in your report. These examples should include both positive and negative samples.

5. Target Labels

Your main solution must be formulated as either an **object detection** problem or a **semantic segmentation** problem.

Option A: Object Detection

The model predicts bounding boxes around the synthetic shapes.

For each image, return a target dictionary similar to:

```
target = {
  "boxes": boxes,    # FloatTensor of shape [N, 4], format: x_min, y_min, x_max, y_max
  "labels": labels,  # LongTensor of shape [N], all synthetic shapes may share label 1
  "image_id": image_id,
  "is_positive": is_positive
}
```

If the image is negative, then `boxes` and `labels` should be empty tensors with valid shapes.

Option B: Semantic Segmentation

The model predicts a binary mask showing pixels occupied by synthetic shapes.

For each image, return a target dictionary similar to:

```
target = {
  "mask": mask,      # LongTensor or FloatTensor of shape [H, W], 1 for synthetic-shape
  "image_id": image_id,
  "is_positive": is_positive
}
```

If the image is negative, the mask should contain only zeros.

Classification-Only Limitation

A classification-only model that predicts only whether a synthetic shape exists is **not sufficient for full credit**. Classification may be used as a baseline or an additional experiment, but the main model should localize the synthetic addition using bounding boxes or segmentation masks.

6. Model Requirement

Train a CNN-based architecture to detect or segment the synthetic shapes.

You may train a model from scratch or fine-tune a pretrained model. You must clearly state which parts of the model are pretrained and which parts are trained by you.

Possible detection models include:

- Faster R-CNN,
- SSD-style detector,
- YOLO-style detector,
- a simplified custom CNN detector.

Possible segmentation models include:

- U-Net,
- FCN-style network,
- encoder-decoder CNN,
- custom CNN segmentation architecture.

Your model does not need to be extremely large. A well-designed, clearly trained, and properly evaluated smaller model is preferable to a large model with unclear methodology.

7. Training Requirements

Your training procedure must include:

- a PyTorch Dataset implementation,
- a PyTorch DataLoader ,
- a clearly defined loss function,

- an optimizer,
- a learning-rate choice,
- a fixed number of epochs or a clear stopping criterion,
- validation-set monitoring,
- saved quantitative results,
- prediction visualizations.

You must report at least the following training details:

- input image size,
- batch size,
- number of epochs,
- optimizer,
- learning rate,
- loss function,
- whether pretrained weights were used,
- hardware used, for example CPU, local GPU, or Google Colab GPU,
- approximate training time.

The test set must not be used for model selection.

8. Baseline Requirement

You must compare your CNN-based model against at least one simple baseline.

Possible baselines include:

- color thresholding,
- edge detection followed by connected components,
- template matching,
- logistic regression on handcrafted features,
- a shallow CNN,
- a simple classifier trained only on image-level labels.

The baseline does not need to perform well. Its purpose is to help you determine whether your neural network is learning something meaningful beyond a simple rule.

9. Evaluation Metrics

Use metrics appropriate for your problem formulation.

Detection Metrics

If you choose object detection, report at least:

- precision at IoU threshold 0.5,
- recall at IoU threshold 0.5,
- F1-score at IoU threshold 0.5,
- mean IoU of matched predictions.

If you implement average precision or mAP, include it as an additional metric.

Segmentation Metrics

If you choose semantic segmentation, report at least:

- foreground IoU / Jaccard score,
- Dice coefficient,
- foreground precision,
- foreground recall.

Pixel accuracy alone is not sufficient, because the foreground synthetic-shape region may occupy only a small part of the image.

Required Visual Evaluation

Your report must include prediction visualizations for at least 12 test images. These should include:

- successful examples,
 - failure cases,
 - positive images,
 - negative images.
-

10. Required Experiments

You must perform at least **three** meaningful experiments.

Possible experiments include:

1. training from scratch vs fine-tuning a pretrained model,
2. small training set vs larger training set,
3. easy synthetic shapes vs hard synthetic shapes,
4. high-opacity shapes vs low-opacity shapes,

5. with data augmentation vs without data augmentation,
6. different CNN architectures,
7. different input resolutions,
8. detection formulation vs segmentation formulation,
9. shallow baseline vs deeper CNN,
10. effect of the number of synthetic shapes per image.

For each experiment, report quantitative results and provide a short interpretation. Do not only list numbers; explain what the numbers suggest.

11. Required Deliverables

Submit the following:

1. source code or a Jupyter Notebook,
2. a short report in PDF or Markdown format,
3. a `README.md` file explaining how to run your code,
4. a `requirements.txt` or environment description,
5. generated sample visualizations,
6. prediction visualizations,
7. result tables,
8. a brief discussion of failure cases.

Do not submit the COCO dataset itself. Your code should assume that COCO is available in the required directory structure.

Your report should include:

- dataset preparation,
 - train/validation/test split,
 - synthetic-shape generation method,
 - label-generation method,
 - model architecture,
 - training details,
 - baseline method,
 - evaluation metrics,
 - experimental results,
 - visual examples,
 - failure-case discussion,
 - limitations and possible improvements.
-

12. Reproducibility Requirements

Your submission must be reproducible.

You must specify:

- Python version,
- PyTorch version,
- torchvision version,
- required packages,
- random seeds,
- directory structure,
- commands needed to run training and evaluation.

At minimum, your repository or submission folder should contain:

```
README.md
requirements.txt
train.py or train.ipynb
evaluate.py or evaluation section in notebook
src/ or helper code files
results/
  metrics.csv or metrics.json
figures/
```

Your code should not require manual labeling or manual editing of generated labels.

13. Restrictions

You may use:

- PyTorch,
- torchvision,
- NumPy,
- Matplotlib,
- scikit-learn,
- PIL/Pillow,
- OpenCV, if needed,
- pretrained CNN backbones.

You may not:

- use an existing synthetic-shape detection dataset,
- manually label the synthetic shapes,
- use the COCO object labels as the target labels,
- tune hyperparameters on the test set,
- submit only image-level classification as the main solution,
- submit results without visual examples,
- submit code that cannot reproduce the synthetic labels.

Use of generative AI tools for writing explanations or debugging must be acknowledged if your university policy requires it. All submitted code and results remain your responsibility.

14. Grading Rubric

Total: **100 points**

A. COCO Dataset Use and Splitting — 15 points

- correctly uses MS COCO 2017: 4
- correctly uses `torchvision.datasets.CocoDetection` : 4
- follows the required train/validation/test split: 4
- avoids train/test leakage: 3

B. Synthetic Data Generation — 20 points

- generates varied synthetic shapes: 4
- generates labels automatically: 5
- includes positive and negative images: 3
- makes the task nontrivial: 5
- visualizes generated samples clearly: 3

C. Model and Training — 20 points

- uses a suitable CNN-based detection or segmentation model: 6
- implements the training pipeline correctly: 5
- uses an appropriate loss function: 3
- reports training details clearly: 3
- handles pretrained or scratch training properly: 3

D. Evaluation and Baseline — 20 points

- uses appropriate detection or segmentation metrics: 6
- evaluates on the fixed test set: 4
- compares against a simple baseline: 4
- provides prediction visualizations: 3
- discusses failure cases: 3

E. Experiments and Analysis — 15 points

- includes at least three meaningful experiments: 6
- reports results in clear tables: 3
- interprets results critically: 4
- discusses limitations honestly: 2

F. Code Quality and Reproducibility — 10 points

- code is organized and readable: 3
 - instructions are clear: 3
 - dependencies and seeds are documented: 2
 - results can be reproduced: 2
-

15. Maximum Credit Limitation for Classification-Only Submissions

A submission whose main model performs only image-level classification can receive at most **50 / 100**, even if the classification accuracy is high.

This is because the main objective of the exam is to find the synthetic additions, not merely to decide whether an image contains one.

16. Suggested Report Structure

Use the following structure for your report:

1. Introduction
2. Dataset and Split
3. Synthetic Shape Generation
4. Label Generation
5. Model Architecture

6. Training Procedure
7. Baseline Method
8. Evaluation Metrics
9. Experiments and Results
10. Prediction Visualizations
11. Failure Cases
12. Conclusion

The report does not need to be long, but it must be precise, reproducible, and supported by figures and tables.

17. Expected Learning Outcomes

After completing this practice exam, you should be able to answer the following questions:

1. How can synthetic labels be generated automatically for a computer-vision task?
2. Can a CNN learn to localize artificial changes added to natural images?
3. How does the difficulty of the synthetic generation process affect performance?
4. Does fine-tuning help compared with training from scratch?
5. Is the learned model better than a simple non-deep-learning baseline?
6. What types of synthetic additions are difficult for the model to detect?
7. How can visual inspection reveal errors that scalar metrics hide?



Original Image



Altered Image (that will be used for model training & testing)